

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра ВТ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Основы машинного обучения»
Тема: «Кластеризация»

Студент гр. 2308

Кислицин К. Р.

Преподаватель

Турсуков Н. О.

Санкт-Петербург

2024

Содержание

Цель работы	3
Задачи работы	3
Теоретическая информация.....	3
Ход работы.....	6
Выводы	17
Приложения	18

Цель работы

Целью работы является получение и закрепление навыков предобработки данных и применения методов машинного обучения для решения задач кластеризации.

Задачи работы

1. Предварительная обработка данных (аналогично л/р 1)
 - Визуализация значимых признаков (диаграммы рассеяния, ящики с усами, гистограммы)
 - Очистка данных (удаление пропусков, нормализация, удаление дубликатов)
 - Корреляция данных (матрица корреляций)
2. Обучение моделей и подбор параметров:
 - К-средних (k-means)
 - DBSCAN
 - Иерархическая кластеризация
3. Оценка моделей
 - Визуализация предсказанных значений
 - Сравнение разбиения на классы с помощью кластеризации с реальными
 - Экспертная оценка

Теоретическая информация

Визуализация значимых признаков

- Диаграммы рассеяния (scatter plots): визуализируют зависимость между двумя количественными признаками, что помогает выявить линейные и нелинейные взаимосвязи, а также обнаружить выбросы.
- Ящики с усами (boxplots): позволяют оценить распределение признаков, медиану, квартильные значения и возможные выбросы в данных.
- Гистограммы (histograms): отображают распределение данных по категориям или интервалам для одного признака, помогают оценить симметрию, наличие выбросов и форму распределения.

Матрица корреляций – квадратная таблица, заголовками строк и столбцов которой являются обрабатываемые переменные, а на пересечении строк и столбцов выводятся коэффициенты корреляции для соответствующей пары признаков.

К-средних (k-means)

- KNN – это популярный алгоритм кластеризации, который группирует данные в k кластеров, минимизируя внутрикластерное расстояние.
- Выбираются случайные k центроидов (центральных точек кластеров) из пространства данных. Каждая точка назначается к кластеру, чей центроид находится ближе всего (по метрике, например, Евклидово расстояние). Новые центроиды рассчитываются как среднее всех точек, принадлежащих к каждому кластеру.
- Подбор параметров: Основной параметр – это число кластеров (k). Также важен выбор метрики расстояния и критерия остановки.

DBSCAN

- DBSCAN – это алгоритм кластеризации на основе плотности.
- Для каждой точки оценивается плотность, основанная на числе соседних точек в радиусе ϵ . Точки делятся на:
 - Основные точки: имеют не менее MinPts соседей в радиусе ϵ .
 - Граничные точки: имеют менее MinPts, но находятся в пределах ϵ от основной точки.
 - Шум: точки, не принадлежащие ни одному кластеру.
- Алгоритм соединяет основные точки и их соседей в единые кластеры, расширяя их, пока плотность остается достаточной.
- Подбор параметров: ключевые параметры — это ϵ (радиус окрестности) и MinPts (минимальное количество точек в радиусе для формирования кластера). Их настройка зависит от плотности данных.

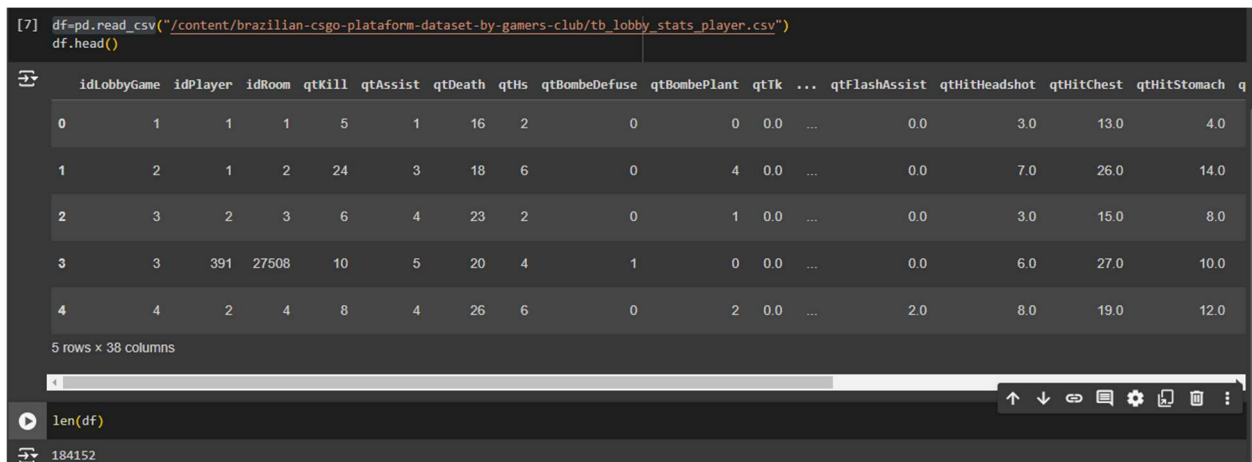
Иерархическая кластеризация

- Случайный лес – это метод кластеризации, который строит древовидную структуру (дендрограмму), объединяя данные по принципу схожести.
- Методы:
 - Агломеративный (снизу вверх): каждая точка сначала рассматривается как отдельный кластер, затем ближайшие кластеры объединяются до тех пор, пока не останется один общий кластер.
 - Дивизивный (сверху вниз): начинается с одного общего кластера, который итеративно разделяется на подгруппы.
- Подбор параметров: ключевые параметры – метрика расстояния (например, Евклидово расстояние, косинусное сходство) и критерий остановки (например, количество кластеров или пороговое значение расстояния для объединения)..

Ход работы

В рамках данной лабораторной работы был рассмотрен датасет по статистике игроков бразильского геймерского клуба в CS:GO. Датасет содержит в себе 184152 строки и 38 признаков.

Сформировал датафрейм на основе загруженных данных и вывел его первые 5 значений (Рисунок 1).



The screenshot shows a Jupyter Notebook cell with the following code and output:

```
[7]: df=pd.read_csv("../content/brazilian-csgo-plataform-dataset-by-gamers-club/tb_lobby_stats_player.csv")
df.head()
```

	idLobbyGame	idPlayer	idRoom	qtKill	qtAssist	qtDeath	qtHs	qtBombeDefuse	qtBombePlant	qtTk	...	qtFlashAssist	qtHitHeadshot	qtHitChest	qtHitStomach	q
0	1	1	1	5	1	16	2	0	0	0.0	...	0.0	3.0	13.0	4.0	
1	2	1	2	24	3	18	6	0	4	0.0	...	0.0	7.0	26.0	14.0	
2	3	2	3	6	4	23	2	0	1	0.0	...	0.0	3.0	15.0	8.0	
3	3	391	27508	10	5	20	4	1	0	0.0	...	0.0	6.0	27.0	10.0	
4	4	2	4	8	4	26	6	0	2	0.0	...	2.0	8.0	19.0	12.0	

5 rows x 38 columns

len(df)
184152

Рисунок 1 – Предосмотр датафрейма

Построил карту пропусков, пропусков не оказалось, потому добавил их искусственно в два столбца. Соответствующие действия показаны на Рисунке 2.

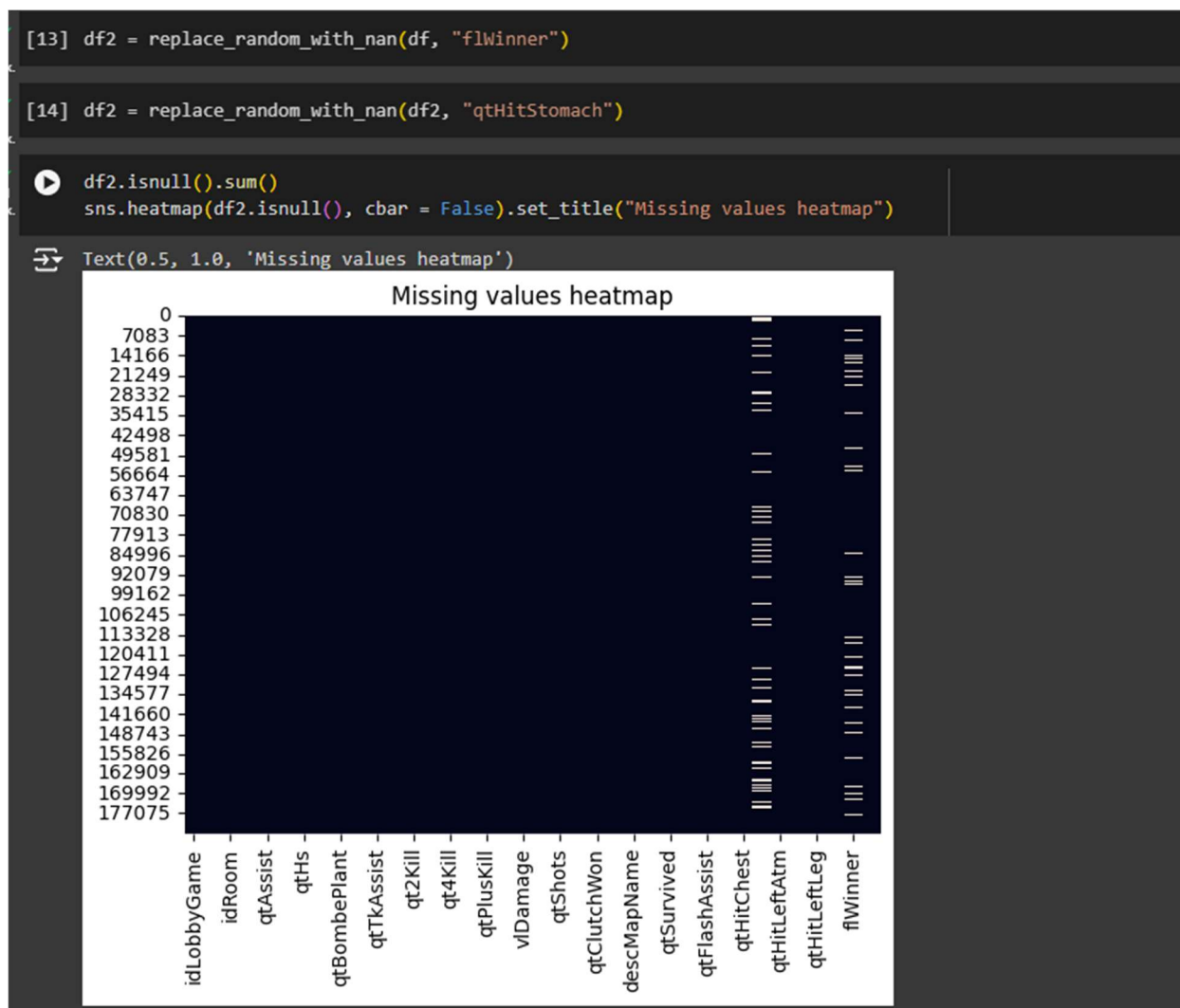


Рисунок 2 – Добавление пропусков

Посмотрел тип данных в столбцах для того, чтобы определить дальнейшие действия по обработке. Нашлось два признака с типом данных «object»: название карты и дата создания записи. Дату разбил на несколько столбцов, название – закодировал. Убрал строки, в которых есть пустые значения для предсказываемого параметра. Все действия приведены на рисунках 3-4.

```
df2.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 184152 entries, 0 to 184151
Data columns (total 38 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   idLobbyGame          184152 non-null  int64
1   idPlayer              184152 non-null  int64
2   idRoom                184152 non-null  int64
3   qtKill                184152 non-null  int64
4   qtAssist              184152 non-null  int64
5   qtDeath               184152 non-null  int64
6   qtHs                  184152 non-null  int64
7   qtBombeDefuse         184152 non-null  int64
8   qtBombePlant          184152 non-null  int64
9   qtTk                  184032 non-null  float64
10  qtTkAssist             184032 non-null  float64
11  qt1Kill                184152 non-null  int64
12  qt2Kill                184152 non-null  int64
13  qt3Kill                184152 non-null  int64
14  qt4Kill                184152 non-null  int64
15  qt5Kill                184152 non-null  int64
16  qtPlusKill             184152 non-null  int64
17  qtFirstKill            184152 non-null  int64
18  vlDamage               184152 non-null  int64
19  qtHits                 184032 non-null  float64
20  qtShots                184152 non-null  int64
21  qtLastAlive            184032 non-null  float64
22  qtClutchWon            184152 non-null  int64
23  qtRoundsPlayed         184152 non-null  int64
24  descMapName            184152 non-null  object
25  vlLevel                184152 non-null  int64
26  qtSurvived             183447 non-null  float64
27  qtTrade                183447 non-null  float64
28  qtFlashAssist          183447 non-null  float64
29  qtHitHeadshot          183447 non-null  float64
30  qtHitChest             183447 non-null  float64
31  qtHitStomach           165089 non-null  float64
32  qtHitLeftArm           183447 non-null  float64
33  qtHitRightArm          183447 non-null  float64
34  qtHitLeftLeg           183447 non-null  float64
35  qtHitRightLeg          183447 non-null  float64
36  flWinner               165737 non-null  float64
37  dtCreatedAt            184152 non-null  object
dtypes: float64(15), int64(21), object(2)
memory usage: 53.4+ MB
```

Рисунок 3 – Вывод типов данных

```
[18] df2 = df2.dropna(subset=['flWinner'])

[21] df2['dtCreatedAt'] = pd.to_datetime(df2['dtCreatedAt'])

df2['year'] = df2['dtCreatedAt'].dt.year
df2['month'] = df2['dtCreatedAt'].dt.month
df2['day'] = df2['dtCreatedAt'].dt.day
df2['dayofweek'] = df2['dtCreatedAt'].dt.dayofweek
df2['hour'] = df2['dtCreatedAt'].dt.hour
df2['quarter'] = df2['dtCreatedAt'].dt.quarter

df2["descMapName"].value_counts()

count
descMapName
de_mirage    52293
de_inferno   37875
de_dust2     21391
de_vertigo   17956
de_overpass  12738
de_nuke      12641
de_train     7033
de_ancient   3810

dtype: int64

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

df2['descMapName'] = le.fit_transform(df2['descMapName'])
df2["descMapName"].value_counts()

Показать скрытые выходные данные

[27] df2.info()

Показать скрытые выходные данные

[26] df2=df2.drop(["dtCreatedAt"], axis=1)
```

Рисунок 4 – Кодирование данных

Дальше построил матрицу корреляции данных (Рисунок 5).

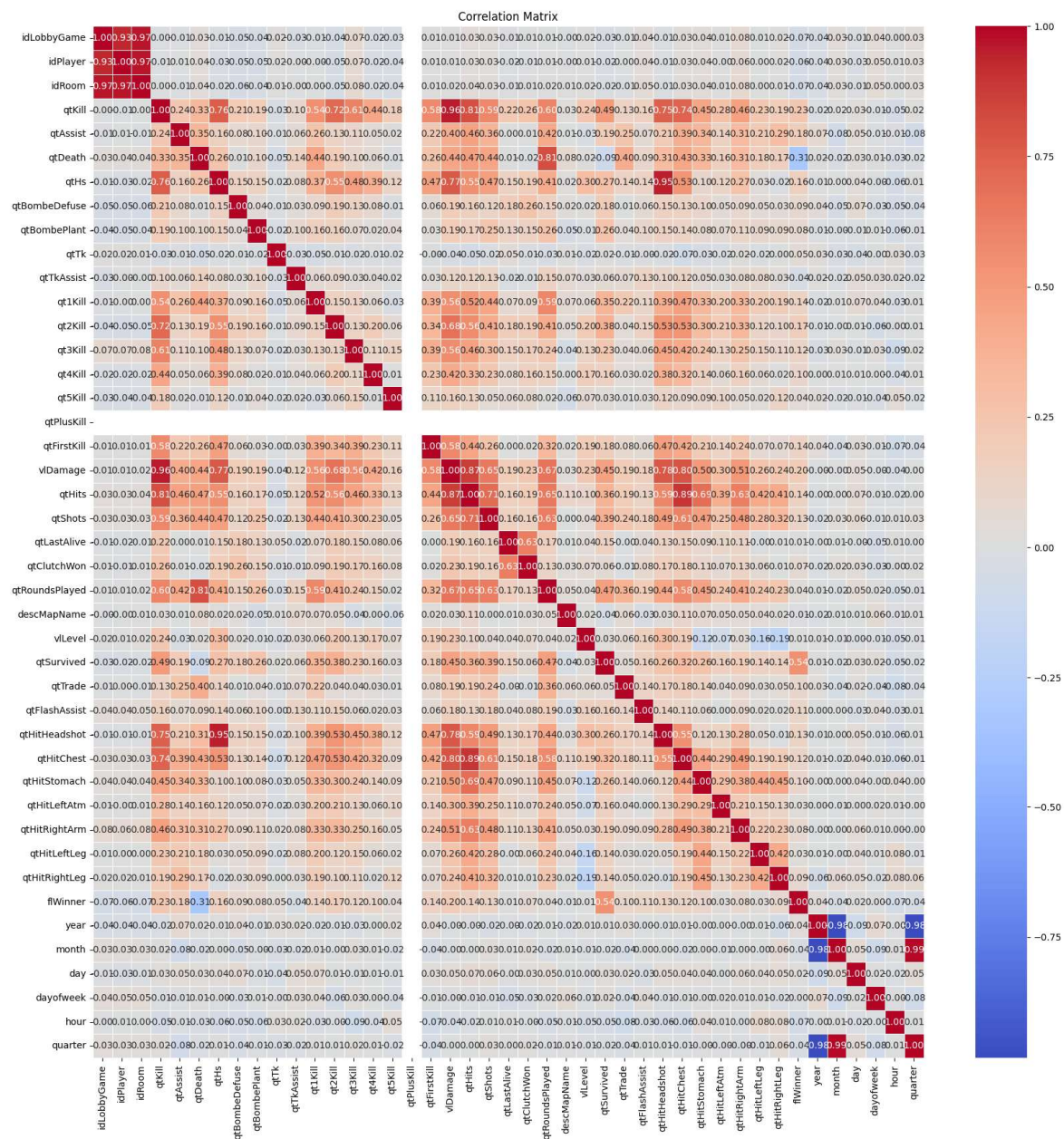


Рисунок 5 – Матрица корреляции данных

По данной матрице были определены параметры, наименее влияющие на другие, у которых коэффициент близок к 0. Удалил эти параметры и заполнил пропуски медианным значением (Рисунок 6).

```
df3 = df3.fillna(df3.median())

[49] df3 = df3.drop(['qtTk', 'day', 'year', 'month', 'qtPlusKill'], axis=1)
```

Рисунок 6 – Заполнение пропусков в столбцах

Построил ящики с усами для определения выбросов в данных (Рисунок 7).

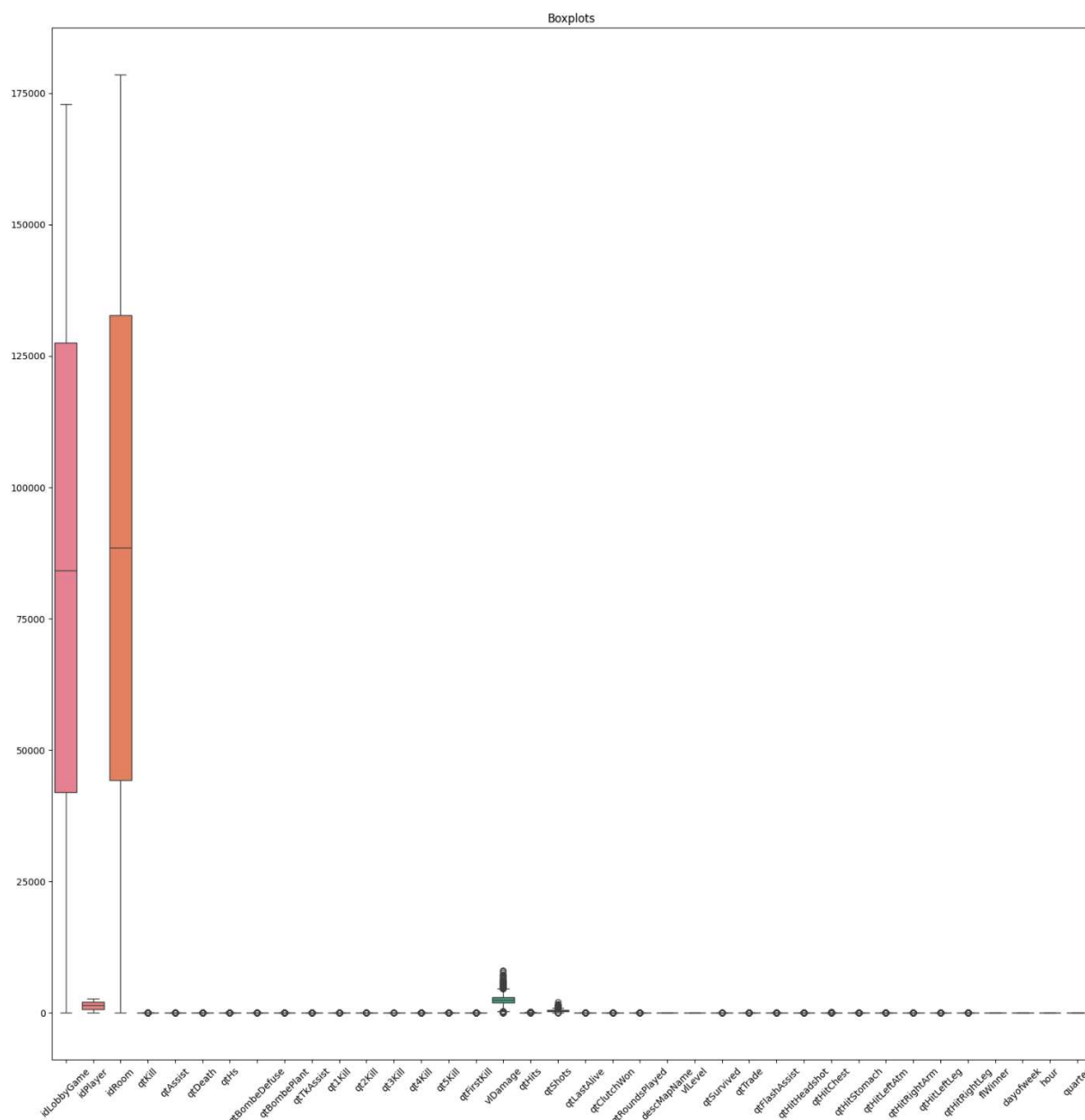


Рисунок 7 – Voxplot для каждого параметра

По графикам видно, что присутствуют статистические выбросы для двух признаков, однако, учитывая контекст данных, они могут иметь сильное влияние на результат (qtFirstKill по крайней мере), потому было принято решение убрать только конкретно видимые выбросы vlDamage (Рисунок 8).

```
[57] df3 = df3[df3['v1Damage'] <= 6500]
```

Рисунок 8 – Чистка сильных выбросов

Дальше выполнил нормализацию признаков и подготовил данные для обучения (Рисунок 9).

```
[58] from sklearn.preprocessing import MinMaxScaler
      scaler = MinMaxScaler()
      df3 = pd.DataFrame(scaler.fit_transform(df3), columns=df3.columns)

[59] x = df3.drop(['flWinner'], axis=1)
      y = df3['flWinner']
```

Рисунок 9 – Завершение предобработки данных

Так как количество данных велико, обучение можно провести на меньшей выборке, с возможностью увеличить её в случае плохих результатов. Выборка представлена на Рисунке 10.

```
[60] x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
```

Рисунок 10 – Выборка данных для обучения

Обучил модель KMeans (Рисунок 13).

```
n_clusters = 7
kmeans = KMeans(n_clusters=n_clusters, random_state=42)
kmeans.fit(df3)
```

KMeans

KMeans(n_clusters=7, random_state=42)

Рисунок 13 – модель KMeans

Количество кластеров пытался подобрать через метод локтя, но не преуспел (Рисунок 14)

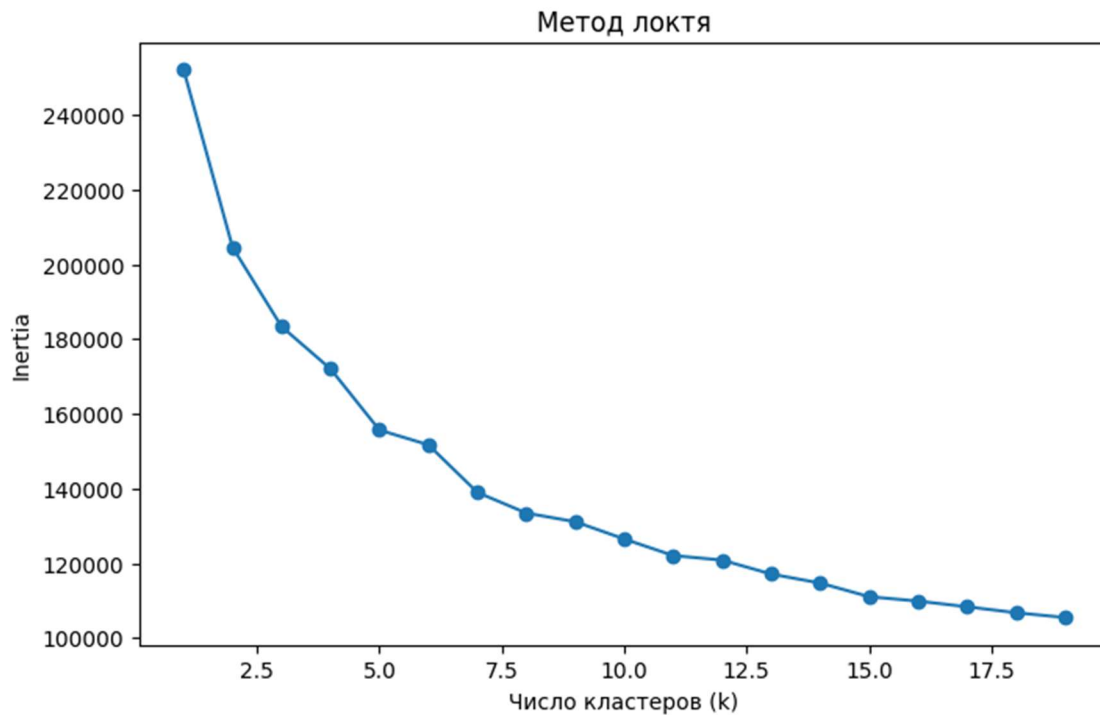


Рисунок 14 – метод локтя

Снизив размерность через UMAP до 2, сделал визуализацию кластеров (Рисунок 15).

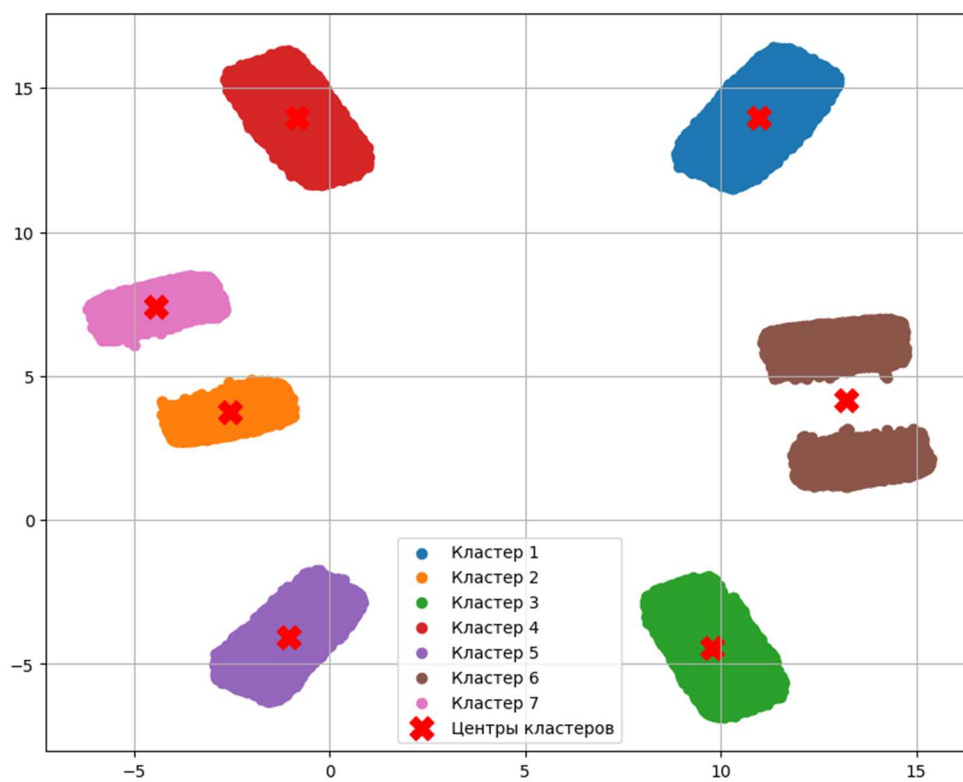
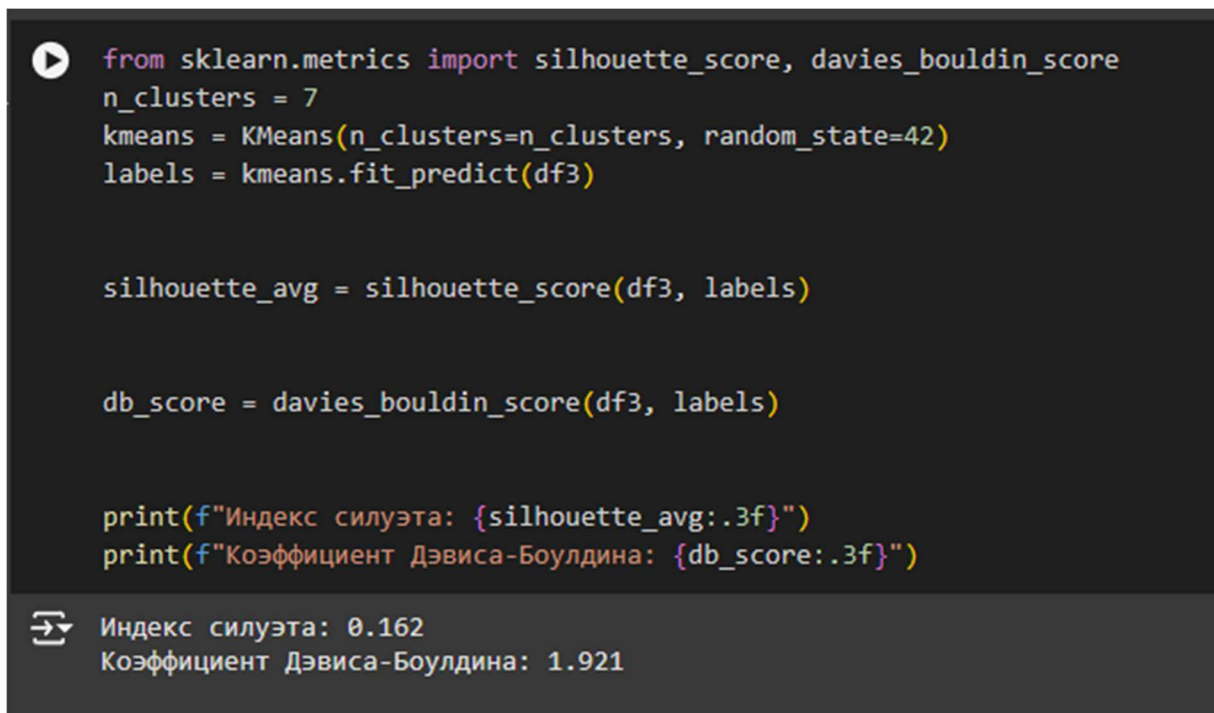


Рисунок 15 – визуализация кластеров

Оценку сравнением провести не представилось возможным т.к. отсутствуют изначальные метки. В качестве альтернативы были посчитаны индекс силуэта (Оценивает, насколько объекты внутри кластера близки друг к другу и насколько они далеки от объектов других кластеров.) и Индекс Дависа-Болдина (Сравнивает внутрикластерную дисперсию и межкластерное расстояние.) (Рисунок 16).



```
from sklearn.metrics import silhouette_score, davies_bouldin_score
n_clusters = 7
kmeans = KMeans(n_clusters=n_clusters, random_state=42)
labels = kmeans.fit_predict(df3)

silhouette_avg = silhouette_score(df3, labels)

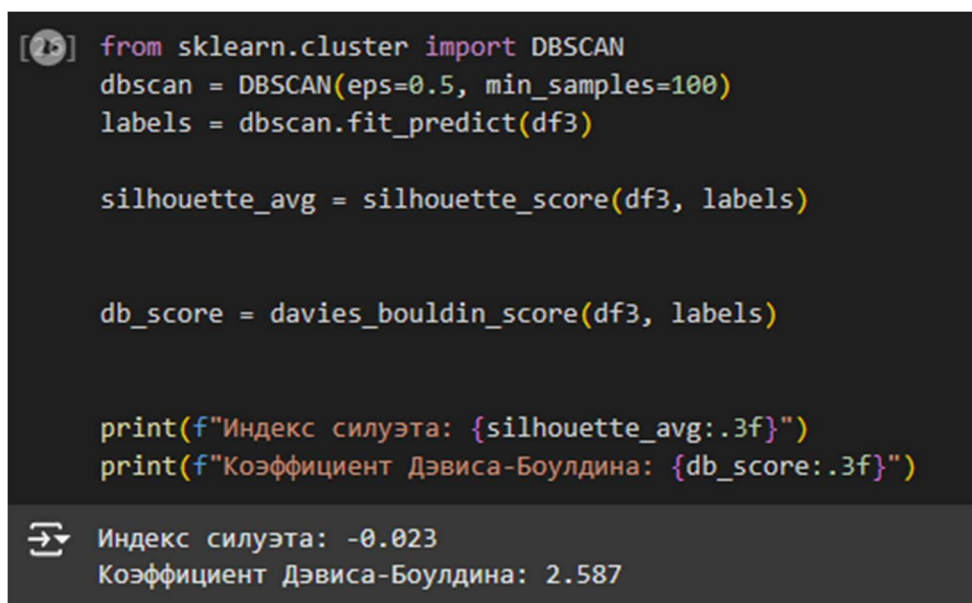
db_score = davies_bouldin_score(df3, labels)

print(f"Индекс силуэта: {silhouette_avg:.3f}")
print(f"Коэффициент Дэвиса-Боулдина: {db_score:.3f}")
```

Индекс силуэта: 0.162
Коэффициент Дэвиса-Боулдина: 1.921

Рисунок 16 – оценка кластеризации

Обучил модель DBSCAN и получил выбранные метрики (Рисунок 17).



```
from sklearn.cluster import DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=100)
labels = dbscan.fit_predict(df3)

silhouette_avg = silhouette_score(df3, labels)

db_score = davies_bouldin_score(df3, labels)

print(f"Индекс силуэта: {silhouette_avg:.3f}")
print(f"Коэффициент Дэвиса-Боулдина: {db_score:.3f}")
```

Индекс силуэта: -0.023
Коэффициент Дэвиса-Боулдина: 2.587

Рисунок 17 – обучение и оценка модели DBSCAN

Провёл визуализацию аналогично предыдущей модели (Рисунок 18).

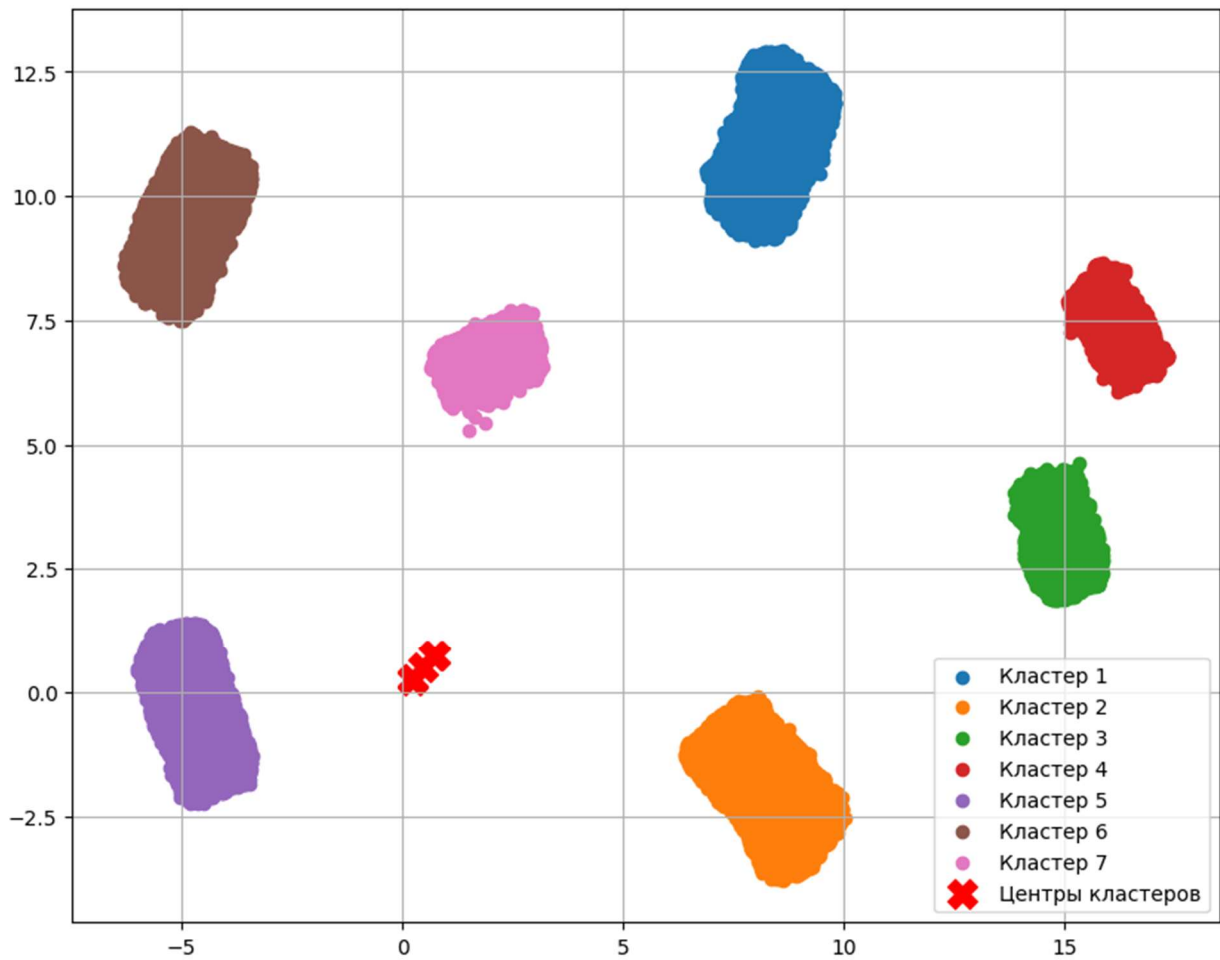


Рисунок 18 – визуализация кластеров

Обучил модель иерархической кластеризации и получил выбранные метрики (Рисунок 19).

```
[30] from sklearn.cluster import AgglomerativeClustering

model = AgglomerativeClustering(n_clusters=7, linkage='ward')
labels = model.fit_predict(df4)

[31] silhouette_avg = silhouette_score(df4, labels)

db_score = davies_bouldin_score(df4, labels)

print(f"Индекс силуэта: {silhouette_avg:.3f}")
print(f"Коэффициент Дэвиса-Боулдина: {db_score:.3f}")

⇒ Индекс силуэта: 0.131
Коэффициент Дэвиса-Боулдина: 2.021
```

Рисунок 19 – обучение и оценка модели иерархической кластеризации

Провёл визуализацию аналогично предыдущей модели (Рисунок 20).

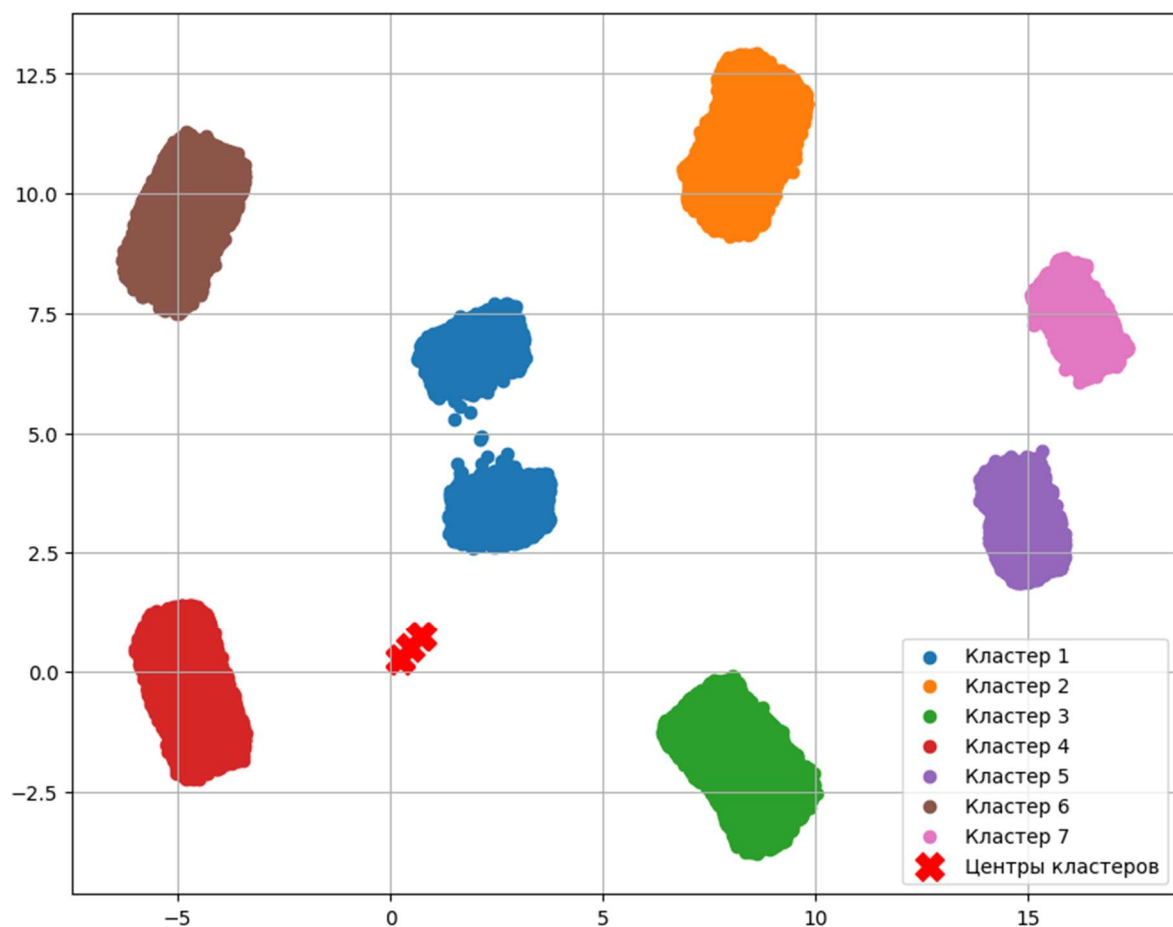


Рисунок 20 – визуализация кластеров

В качестве дополнительной визуализации построил дендограмму (Рисунок 21).

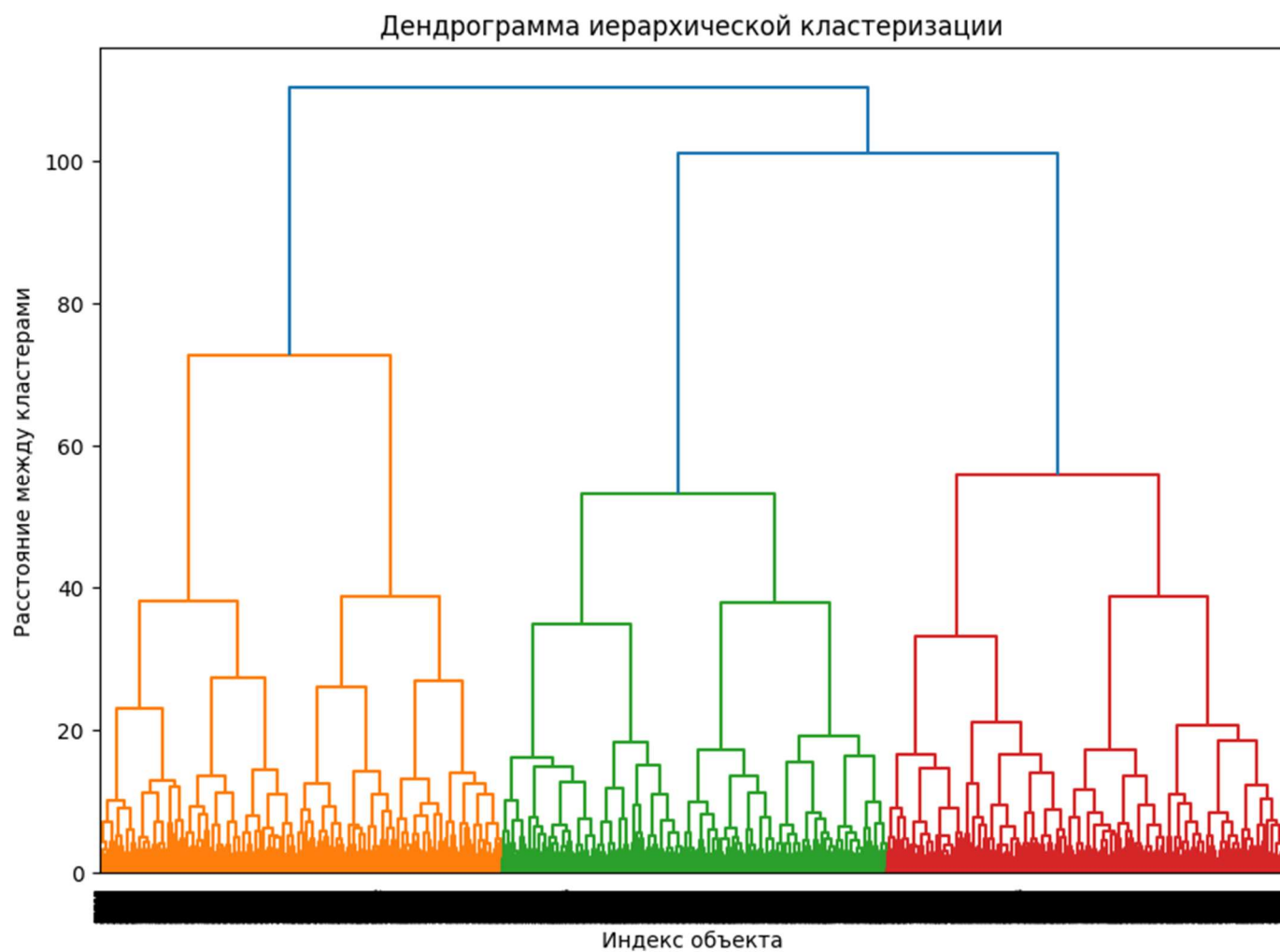


Рисунок 21 – дендограмма

Выводы

В рамках данной лабораторной работы была проведена предварительная обработка данных аналогичная обработке в предыдущей работе. Этот процесс включал визуализацию значимых признаков, очистку данных от пропусков, дубликатов и аномалий, а также нормализацию, что способствовало повышению качества обучения моделей. Анализ корреляций позволил выявить избыточные признаки и избежать проблемы мультиколлинеарности.

На этапе построения моделей были использованы три популярных метода: K-средних, DBSCAN и иерархическая кластеризация. Каждый из этих алгоритмов имеет свои особенности и требует тщательной настройки параметров для достижения оптимальных результатов.

Оценка качества моделей проводилась с использованием метрик индекс силуэта и Индекс Дависа-Болдина, что позволило оценить их точность и полноту в решении задачи классификации.

В целом качество получилось низким, что связано в основном со спецификой данных.

Для улучшения метрик всех моделей, можно увеличить количество данных, провести тонкую настройку параметров для каждой из моделей. Однако это требует много времени и вычислительных ресурсов.

Приложения

Ссылка на блокнот в google.colab: <https://colab.research.google.com/drive/1IDH6uOQUHP6RHGN4-rh3WHVMjTEQP3r#scrollTo=3470ffBBrNgJ>